UNIVERSIDAD DON BOSCO

EmprendeLink

Etapa 3 - Foro de Discusión 1

Avance móvil con React Native y Expo

Asignatura

Diseño y Programación de Software Multiplataforma (DPS941)

Integrantes

Marvin Francisco Pérez Calderón (PC253641)
Rafael Mena Mejia (MM253045)
Tito Mauricio Nochez Villagran (NV101005)
Luis Miguel Granados Artiga (GA130557)
Jorge Alfonzo Mendoza Padilla (MP241100)

Docente

[Completar nombre antes de entregar]

17 de octubre de 2026

DOCUMENTO TÉCNICO

1. Introducción

EmprendeLink es una plataforma académica orientada a conectar microemprendedores con clientes. La aplicación web existente permite administrar negocios, catálogos, productos, pedidos y comisiones con Next.js y Firebase. Esta etapa incorpora una aplicación móvil en React Native, sin alterar el funcionamiento de la web y sin adelantar la integración Firebase exigida para el siguiente foro.

El avance móvil implementa autenticación temporal, navegación funcional y la experiencia principal de cliente: descubrir negocios, consultar catálogos, revisar productos, administrar un carrito, crear un pedido temporal, consultar pedidos y revisar el perfil. La arquitectura conserva la separación UI → lógica → servicios → datos para que los servicios mock puedan reemplazarse por Firebase y Route Handlers sin rehacer las pantallas.

2. Objetivos

Objetivo general

Desarrollar un avance móvil funcional y defendible de EmprendeLink con React Native y Expo, coherente con la identidad visual web y preparado para una integración posterior con Firebase.

Objetivos específicos

1. Implementar login y registro con validaciones, errores y estados de carga.
2. Construir navegación stack y bottom tabs sin pantallas aisladas ni acciones muertas.
3. Adaptar la experiencia cliente web a patrones apropiados para teléfonos.
4. Centralizar componentes visuales, tokens, contratos y datos temporales.
5. Verificar la aplicación mediante pruebas unitarias y un bundle Android de producción.
6. Documentar decisiones, diferencias y limitaciones con fuentes oficiales bajo APA 7.

3. ¿Qué es React Native?

React Native es un framework de código abierto para crear aplicaciones Android e iOS mediante React y JavaScript. Sus componentes describen la interfaz con JSX, pero se respaldan en vistas de la plataforma en lugar de elementos HTML. Entre los componentes esenciales se encuentran View, Text, Image, TextInput, ScrollView y Pressable (Meta Platforms, Inc., s. f.).

Para EmprendeLink esto permite reutilizar el conocimiento del equipo en React, estado y componentes, a la vez que se adoptan interacciones móviles como tabs inferiores, gestos de regreso, teclado virtual, áreas seguras y targets táctiles. No se copió el HTML/CSS del sitio web: se tradujeron sus decisiones de producto a componentes y estilos propios de React Native.

4. Arquitectura de React Native

React Native combina componentes React con capacidades nativas de Android e iOS. La arquitectura moderna del framework mejoró la comunicación con la plataforma, la programación de actualizaciones y el renderizado. Para una aplicación de negocio como EmprendeLink, el valor práctico es mantener la lógica en JavaScript mientras la interfaz se materializa con componentes nativos y puede acceder a módulos de plataforma cuando sea necesario (Meta Platforms, Inc., 2024).

La arquitectura interna del framework no reemplaza la arquitectura de la aplicación. EmprendeLink mantiene sus propias responsabilidades por capas:

```
Screens / components
        ↓ eventos y propiedades
Context / hooks / utils
        ↓ operaciones de dominio
Services
        ↓ respuesta { success, data, error }
Mock data (Foro 1) → Firebase / API segura (Foro 2)
```

Esta decisión evita acceso a datos dentro de JSX y conserva una frontera concreta para la futura persistencia.

5. Expo y su uso en EmprendeLink

Expo es un framework y conjunto de herramientas que simplifica la creación, ejecución y compilación de proyectos React Native. La documentación oficial recomienda create-expo-app para inicializar proyectos y npx expo start para abrir el servidor de desarrollo, mostrar un QR y ejecutar la app en dispositivo o emulador (Expo, s. f.-a, s. f.-b).

EmprendeLink Mobile utiliza Expo SDK 57 con plantilla JavaScript. Se eligió esa versión porque la documentación oficial la identifica como la opción compatible con Expo Go durante la transición a SDK 58 vigente al preparar esta entrega. La aplicación conserva un package-lock.json propio dentro de mobile/, por lo que la web y el móvil pueden instalarse y validarse de forma independiente.

Comandos principales:

```
cd mobile
npm ci
npm start
npm test
npm run export:android
```

6. Navegación móvil

La navegación combina dos patrones. Un stack de autenticación contiene Login y Registro. Después de iniciar sesión, un stack de aplicación contiene las tabs principales y los flujos de detalle. Las tabs inferiores muestran Negocios, Pedidos y Perfil; React Navigation describe este patrón como una barra que permite cambiar entre rutas principales, con inicialización diferida de pantallas (React Navigation, s. f.).

```
Auth Stack
├── Login
└── Registro

App Stack
├── Main Tabs
│   ├── Negocios
│   ├── Pedidos
│   └── Perfil
├── Catálogo
├── Detalle de producto
├── Carrito
└── Detalle de pedido
```

Las tabs representan destinos frecuentes. Catálogo, producto, carrito y detalle de pedido se mantienen en stack porque forman recorridos contextuales y deben conservar navegación de regreso.

7. Componentes reutilizables

La aplicación evita una App.js monolítica. App.js solamente compone providers, navegación y status bar. Los elementos repetidos se encapsularon en:

- AppButton: variantes, loading, disabled y feedback de presión.
- AppInput y PasswordInput: labels, errores y visibilidad de contraseña.
- ScreenContainer: áreas seguras, scroll y adaptación al teclado.
- BusinessCard, ProductCard y OrderCard: representación consistente del dominio.
- LoadingState, EmptyState y ErrorState: estados explícitos de la interfaz.
- QuantityControl, StatusBadge, SearchInput y BrandLockup.

Los componentes reciben datos y callbacks; no conocen los mocks ni Firebase. Esto reduce duplicación, facilita pruebas y permite explicar cada responsabilidad de forma aislada.

8. Validación de formularios

Las reglas viven en src/utils/validation.js, separadas de Login y Registro. El formulario de login valida presencia y formato del correo y presencia de contraseña. El registro valida nombre, correo, contraseña de al menos ocho caracteres con letra y número, confirmación coincidente y, para CUSTOMER, teléfono y dirección. ENTREPRENEUR conserva el flujo web actual y no solicita datos de entrega.

La UI presenta mensajes junto al campo, un error general cuando el servicio rechaza credenciales y un indicador de carga que deshabilita envíos repetidos. PasswordInput permite mostrar u ocultar el valor. Las pruebas unitarias cubren escenarios válidos, campos requeridos, requisitos por rol y coincidencia de contraseñas.

9. Diseño UX/UI

El sistema visual reutiliza la identidad observable en la experiencia cliente web actual:

- azul principal #2563EB y azul oscuro #1D4ED8;
- fondo claro #F4F7FB y superficies blancas;
- texto principal #162033 y texto secundario #667085;
- radios de 8, 12 y 18 puntos;
- espaciado basado en 4, 8, 12, 16, 24 y 32 puntos;
- sombras suaves y estados semánticos de éxito, advertencia y error.

Se utilizaron SafeAreaView, KeyboardAvoidingView, ScrollView, FlatList, Pressable, TextInput e Image según la naturaleza del contenido. Las listas emplean FlatList; formularios y detalles usan scroll; los botones importantes alcanzan aproximadamente 48–50 puntos. Labels de accesibilidad, contraste, fallbacks de imagen, estados vacíos y feedback de presión mejoran la comprensión y operación básica. React Native recomienda FlatList para listas largas porque solo renderiza los elementos visibles (Meta Platforms, Inc., s. f.).

10. Uso de Figma

Figma permite definir flujos mediante frames, conexiones y puntos de inicio para previsualizar el recorrido de una persona antes de desarrollar. Sus prototipos sirven para revisar interacciones, compartir ideas, recibir comentarios y probar recorridos (Figma, Inc., s. f.).

No se encontró en el repositorio un enlace, archivo o permiso verificable al Figma original. Por esa razón, este documento no afirma que se inspeccionó ese archivo. La referencia visual disponible fue la implementación web actual y sus componentes CustomerShell, ProductCard, ProductDetailModal, carrito, pedidos y perfil. Cuando el equipo adjunte el enlace de Figma deberá realizar una revisión visual final y registrar diferencias reales.

11. Aplicación concreta a EmprendeLink

El recorrido principal implementado es:

```
Login → Negocios → Catálogo → Producto → Carrito → Confirmar pedido
                └──────────────────────────────────┐
                                                   ↓
                                        Pedidos → Detalle
```

El usuario puede buscar negocios, filtrar productos por texto y categoría, cambiar cantidades y crear un pedido temporal. El carrito impide mezclar productos de dos negocios sin confirmación, porque el contrato real del pedido contiene un único businessId. Los pedidos usan exclusivamente los estados PENDING, CONFIRMED, PREPARING, READY, DELIVERED y CANCELLED.

12. Arquitectura móvil implementada

```
mobile/src/
├── screens/auth        Login y Registro
├── screens/app         Negocios, catálogo, producto, carrito, pedidos y perfil
├── components          UI reutilizable
├── navigation          Root stack y bottom tabs
├── context             Auth, Cart y Orders
├── hooks               carga de negocios y catálogo
├── services            auth, catálogo y pedidos
├── data                mockData centralizado
├── constants           roles y estados
├── utils               validación y formato
└── theme               tokens visuales
```

authService mantiene credenciales solo en memoria y devuelve perfiles sin contraseña. Los mocks respetan los nombres de campos documentados para User, Business, Category, Product, Order y Order Item. orderService produce un pedido PENDING con snapshot de nombre, precio, plan y tasa, igual que exige el contrato web.

13. Flujo de navegación

Al abrir la app se muestra Login. El acceso válido o el registro actualizan AuthContext, lo que cambia automáticamente al App Stack. Negocios es el destino inicial. Desde una tarjeta se abre el catálogo; desde un producto se abre su detalle; agregar conduce al carrito. Confirmar crea un pedido temporal, limpia el carrito y ofrece abrir Pedidos. Perfil permite cerrar sesión y regresar al Auth Stack. Android y el encabezado del stack ofrecen navegación hacia atrás en pantallas secundarias.

14. Comparación diseño → implementación

Aspecto	Figma original	Web actual	React Native
Acceso	No disponible para verificación	Inspeccionado en el repositorio	Implementado y verificable
Navegación	Pendiente de contraste	Barra responsive de cliente	Bottom tabs + stack
Negocios	Pendiente de contraste	Tarjetas, búsqueda y estados	FlatList, tarjetas y búsqueda
Producto	Pendiente de contraste	Modal de detalle	Pantalla de detalle móvil
Carrito	Pendiente de contraste	Drawer/página	Pantalla con resumen fijo
Pedidos	Pendiente de contraste	Lista y detalle	Tab, cards y detalle en stack
Identidad	Pendiente de contraste	Azul, superficies y bordes suaves	Tokens derivados de la web

La principal diferencia intencional es sustituir patrones de escritorio como modal y drawer por pantallas navegables, que ofrecen mejor espacio, regreso nativo y manejo del teclado en teléfonos.

15. Capturas de evidencia

> *CAPTURA 1 — Login: insertar captura real mostrando correo, contraseña y cuenta demo.*

> *CAPTURA 2 — Registro: insertar captura real con selector de rol y validaciones visibles.*

> *CAPTURA 3 — Negocios: insertar captura real de búsqueda y tarjetas.*

> *CAPTURA 4 — Catálogo: insertar captura real de categorías y productos.*

> *CAPTURA 5 — Detalle y carrito: insertar una o dos capturas del flujo de compra.*

> *CAPTURA 6 — Pedidos: insertar captura del pedido creado y su detalle.*

> *CAPTURA 7 — Perfil: insertar captura del perfil y navegación inferior.*

Las capturas deben provenir del mismo dispositivo o emulador, usar escala legible y no mostrar credenciales reales.

16. Decisiones técnicas

1. Monorepo simple: mobile/ mantiene aisladas dependencias y lockfile sin alterar Next.js.
2. JavaScript: respeta la decisión tecnológica existente y evita una migración no autorizada.
3. React Navigation: tabs para destinos principales y stack para recorridos jerárquicos.
4. Servicios reemplazables: ninguna pantalla importa mocks directamente.
5. Context API: conserva el patrón conocido por el equipo para sesión, carrito y pedidos.
6. Carrito de un negocio: refleja el único businessId del contrato Order.
7. Sin secretos: no se copiaron variables, cuentas de servicio ni credenciales Firebase.
8. Sin escritura real: el avance no toca datos productivos y respeta el alcance de Foro 1.

17. Ventajas y limitaciones

Ventajas

- recorrido completo demostrable en un teléfono;
- UI consistente, componentes pequeños y nombres defendibles;
- validación separada y cubierta por pruebas;
- mocks realistas alineados con contratos compartidos;

- sustitución futura de services sin rediseñar pantallas;
- bundle Android verificable y web existente sin modificaciones funcionales.

Limitaciones

- sesión, cuentas y pedidos se pierden al reiniciar;
- no hay Firebase Authentication, Google Sign-In ni Firestore móvil;
- el rol emprendedor puede registrarse, pero su panel móvil no forma parte de Foro 1;
- las imágenes de demostración son remotas y requieren internet;
- falta comparar píxel a píxel con Figma porque no se encontró acceso al archivo;
- las capturas y el video requieren ejecución visual por el equipo.

18. Trabajo futuro: Firebase y Foro 2

Foro 2 deberá reemplazar authService por Firebase Authentication y cargar el documento users; reemplazar catalogService por consultas de lectura permitidas; y conectar orderService con Route Handlers. La app móvil no debe escribir directamente orders, orderHistory o commissions. Identidad, rol, propiedad, precios y transiciones deben validarse en servidor mediante Firebase Admin. También se recomienda persistir sesión y carrito, añadir recuperación de contraseña, probar dispositivos de distintos tamaños y completar el panel móvil del emprendedor si la rúbrica lo exige.

19. Verificación realizada

La aplicación móvil fue verificada mediante cinco pruebas unitarias de validación y una exportación Android de producción con Metro. La exportación resolvió 949 módulos y generó el bundle sin imports rotos. El proyecto web debe validarse antes de cerrar la entrega con npm test y npm run check. Las evidencias exactas y fecha de ejecución final se registran en el checklist del repositorio.

20. Conclusiones

El avance cumple el objetivo de presentar una aplicación móvil funcional sin sobrepasar el alcance de Foro 1. La solución no es una maqueta estática: permite autenticarse, registrarse, navegar, filtrar, agregar productos, confirmar un pedido temporal, consultarlo y cerrar sesión. La separación entre UI, estado, servicios y datos reduce el costo de incorporar Firebase en la siguiente etapa.

La adaptación conserva la identidad actual de EmprendeLink, pero utiliza patrones propios del teléfono. La principal evidencia pendiente depende del equipo: capturas reales, contraste final con el archivo Figma autorizado y grabación de un video máximo de 15 minutos usando la app ejecutándose.

21. Referencias

Expo. (s. f.-a). Create a project. Recuperado el 17 de octubre de 2026, de https://docs.expo.dev/get-started/create-a-project/

Expo. (s. f.-b). Start developing. Recuperado el 17 de octubre de 2026, de https://docs.expo.dev/get-started/start-developing/

Figma, Inc. (s. f.). Guide to prototyping in Figma. Recuperado el 17 de octubre de 2026, de https://help.figma.com/hc/en-us/articles/360040314193-Guide-to-prototyping-in-Figma

Meta Platforms, Inc. (s. f.). Core components and APIs. React Native. Recuperado el 17 de octubre de 2026, de https://reactnative.dev/docs/components-and-apis

Meta Platforms, Inc. (2024, 23 de octubre). The new architecture is here. React Native. https://reactnative.dev/blog/2024/10/23/the-new-architecture-is-here

React Navigation. (s. f.). Bottom tabs navigator. Recuperado el 17 de octubre de 2026, de https://reactnavigation.org/docs/bottom-tab-navigator/